

Uncertainty-Aware Error Detection: Mechanistic Interpretability

Troy Daniels

November 2025

1 Introduction

I hypothesize that specific internal activations are the best indicators of an LLM’s confidence. In contrast, logits and derived probabilities often overestimate sequence likelihood, even when the model is explicitly prompted to be fair or deliberately conservative about what it doesn’t know, making them untrustworthy for UQ.

A better estimate may lie in the model’s internal state. This rests on the assumption that detecting true features supporting correctness is the most direct and honest basis for confidence. For example, in identifying an animal species from an image, articulating the features that imply the species is more trustworthy than merely stating the species.

Accepting this assumption, the key questions are: (1) whether there exist activations representing task-relevant features, (2) whether they can be reliably identified, and (3) how to aggregate and measure them to yield accurate confidence scores during generation. I aim to devise a process to efficiently find these activations, cluster them by scope/task, and measure them to produce calibrated confidence estimates. Or if proven too in-depth considering time constraints, finding a way to best approximate this using prompts.

2 Mechanistic Interpretability through Entropy Neurons

2.1 Entropy Neurons Intro

Entropy neurons are a newly identified type of ...

2.2 Finding Entropy Neurons through Logit Attribution

2.2.1 Logit Attribution

The final pre-softmax step of a transformer is linear, $\ell = W_U r$, where $W_U \in \mathbb{R}^{V \times d_{\text{model}}}$ is the unembedding matrix, $r \in \mathbb{R}^{d_{\text{model}}}$ is the residual stream, and $\ell \in \mathbb{R}^V$ are the logits. If we decompose the residual as an exact sum of components $r = \sum_k c_k$ (e.g., contributions from specific attention heads, MLP neurons, or earlier positions), then the logits decompose additively: $\ell = \sum_k W_U c_k$. The contribution of component k to the logit of token t is the inner product $\Delta \ell_t^{(k)} = \langle W_U[t, :], c_k \rangle$, and summing over k recovers the full logit ℓ_t . This *direct logit attribution* yields precise per-token, per-component attributions for the final linear map, while noting that nonlinearities and LayerNorm effects earlier in the network are not captured by this last-step linear decomposition.

2.3 Intuition Behind the Method

Define:

MLP = Multi-Layer Perceptron

$|V|$ = Cardinality of the Vocabulary

$\mathbf{W}_{in} \in \mathbb{R}^{d_{mlp} \times d_{model}}$ = Learned Weight Matrix

$\mathbf{W}_{out} \in \mathbb{R}^{d_{model} \times d_{mlp}}$ = Learned Weight Matrix

$\mathbf{W}_U \in \mathbb{R}^{|V| \times d_{model}}$ = Projection to the Vocabulary Space (Unembedding Matrix $\rightarrow$ Logits)

σ = activation function

β_{in}, β_{out} = Learned Biases

$\mathbf{x} \in \mathbb{R}^{d_{model}}$ = Normalized Residual Stream Hidden State

Where the structure of a MLP layer of the transformer is:

$$MLP(\mathbf{x}) = \mathbf{W}_{out} \sigma(\mathbf{W}_{in} \mathbf{x} + \beta_{in}) + \beta_{out}$$

$$W_{in} X_0 \rightarrow \begin{matrix} & W_{in} & & & \\ & & & X_0 & \\ & & & & x_1 \end{matrix} \begin{bmatrix} | & | & \dots & | \\ v_1 & v_2 & \dots & v_{d_{model}} \\ | & | & \dots & | \end{bmatrix} \begin{bmatrix} x_0 \end{bmatrix} = \begin{bmatrix} x_1 \end{bmatrix}$$

Assume biases and activation function happen implicitly

$$W_{out} X_1 \rightarrow \begin{matrix} & W_{out} & & & \\ & & & x_1 & \\ & & & & \end{matrix} \begin{bmatrix} | & | & \dots & | \\ u_1 & u_2 & \dots & u_{d_{MIP}} \\ | & | & \dots & | \end{bmatrix} \begin{bmatrix} x_1 \end{bmatrix} = \begin{bmatrix} x_2 \end{bmatrix}$$

Ignoring the Layer Norm

$$W_U X_2 \rightarrow \begin{bmatrix} | & | & \dots & | \\ z_1 & z_2 & \dots & z_{d_{model}} \\ | & | & \dots & | \end{bmatrix} \begin{bmatrix} x_2 \end{bmatrix} = \begin{bmatrix} v \end{bmatrix} \text{ (logits)}$$

But Notice, (let $d_1 = \text{constant}$):

$$\begin{bmatrix} x_2 \end{bmatrix} = d_1 u_1 + d_2 u_2 + \dots + d_{d_{MIP}} u_{d_{MIP}}$$

which means:

$$W_U X_2 \rightarrow W_U d_1 u_1 + W_U d_2 u_2 + \dots + W_U d_{d_{MIP}} u_{d_{MIP}}$$

where

$$W_U d_i u_i \in \mathbb{R}^{|V| \times 1} \text{ (One term in a sum for the logits)}$$

2.3.1 Finding Entropy Neurons

Entropy neurons are found in the last layer of an autoregressive model, just before the final LayerNorm and the unembedding layer (which maps hidden states to vocabulary logits). The steps to identify entropy neurons are:

1. Retrieve the last weight matrix before the LayerNorm and unembedding. Denote it by W_{out} .
2. Treat each column of W_{out} as a single neuron. Let w_i be the i -th column (the i -th neuron).
3. Retrieve the unembedding matrix W_U .
4. For each w_i , compute $\psi_i = W_U w_i$. This gives the per-token logit effect of that neuron (per unit activation).
5. Normalize ψ_i by dividing by the column norms of W_U and by $\|w_i\|$ to remove scale effects.
6. Compute the variance of the normalized ψ_i .
7. If $\|\psi_i\|$ is high and the variance of the normalized ψ_i is low, mark neuron i as an entropy neuron.

High magnitude and low variance suggest an entropy neuron: low variance means the neuron shifts most logits by a similar amount (leaving the winner largely unchanged), and high magnitude indicates a stronger overall effect on confidence (entropy), even under weight decay.

2.4 Using Entropy Neurons

Once the Entropy Neurons are identified, they can be used to estimate uncertainty. Providing a prompt and recording the activation for each neuron (α_i) will approximate the effect of that neuron on the logits. Considering there will be more than one Entropy Neuron the final estimate can be:

1. Sum of the activations over the whole generation.
2. Sum of the activations for last word predicted.
3. Max activation over the whole generation.
4. Max activation for last word predicted.
5. Average activation over the whole generation.
6. Average activation for last word predicted.

References